

Fundamentals of Software Systems

Towards Scalable NoSQL Data Management

Christoph Lofi

Note

Data Management (and especially distributed Data Management) is a **very large and complex topic**. A few lectures -as in this course- is not enough to teach you how to implement (or even use) these systems competently. Instead, these lectures try to scratch the surface of many different topics. Your main takeaway should be that

- a) data management is a **hard problem**
- b) there are no cookie-cutter solutions (**it's always just compromises**)
- c) **you still know very little about this field** (and that's OK, just be aware of this and **don't underestimate** the topic, and be prepared to do more **self-studies in the future.**)

Relational Databases are Awesome

ORACLE®



PostgreSQL



Relational Databases are Awesome

- Traditional databases are usually **all-purpose systems**
 - e.g. DB2, Oracle, MySQL, ...
 - Theoretically, general purpose DB provide all features to develop any data driven application
- **Powerful query languages**
 - SQL, can be used to **update** and **query** data; even **very complex analytical queries** possible
- **Expressive data model**
 - Most data modeling needs can be served by the **relational model**



ORACLE®



Relational Databases are Awesome

- **Full transaction support**
 - Transactions are guaranteed to be “safe”
 - i.e. ACID transaction properties
- **Atomicity**
 - any transaction is either executed **completely** or **not at all**
- **Consistency (preservation)**
 - transactions lead from one **consistent state** of the data instance to another
- **Isolation**
 - transactions are isolated from others, i.e., even in a concurrent scenario **transactions do not interfere** with each other
- **Durability**
 - as soon as the transaction is completed (committed), all **data changes** performed are guaranteed to **survive** subsequent system failures



Relational Databases are Awesome

- **System durability and security**
 - Database servers are **resilient to failures**
 - **Log files** are continuously written
 - Transactions running during a failure can **recovered**
 - Most databases have support for constant **backup**
 - Even severe failures can be recovered from backups
 - Most databases support “**hot-standby**”
 - 2nd database system running simultaneously which can take over in case of severe failure of the primary system
 - Most databases offer basic **access control**
 - i.e. **authentication** and **authorization**









???



Towards Non-Relational Systems

- **Relational Databases** might face problems in Extreme Conditions
 - Problems when being faced with **massively** high query loads
 - i.e. millions of transactions per second
 - Load too high for a single machine or even a traditional distributed database
 - Limited scaling
 - Problems with fully **global applications**
 - Transactions originate from all over the globe
 - **Latency matters!**
 - Data should be geographically close to users
 - **Claims:**
 - Amazon: increasing the latency by 10% will decrease the sales by 1%
 - Google: increasing the latency by 500ms will decrease traffic by 20%

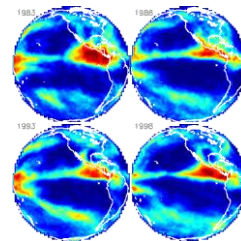
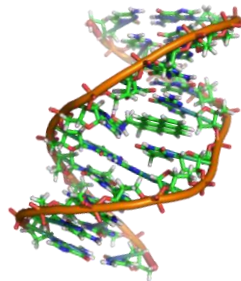


Towards Non-Relational Systems

- Problems with extremely high **availability** constraints
 - Traditionally, databases can be recovered using logs or backups
 - Hot-Standbys may help during repair time
 - But for some applications, this is not enough:
Extreme Availability (Amazon)
 - “... must be available even if disks are failing, network routes are flapping, and several data centers are destroyed by massive tornados”
 - Additional availability and durability concepts needed!

Towards Non-Relational Systems

- Problems with **emerging applications** requiring **new data models**
 - Traditional databases rely on the **relational model** which is not optimal for many new applications
 - e.g. scientific data management like genome databases, geo-information databases, etc.
 - e.g. for handling data streams and massive volumes of sensor data
 - e.g. for handling knowledge networks and reasoning



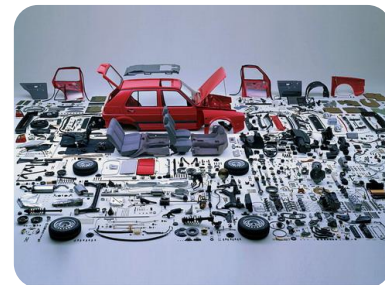
Towards Non-Relational Systems

- In extreme cases, specialized database-like systems may be beneficial
 - Specialize on certain query types
 - **Focus on a certain characteristic**
 - i.e. availability, scalability, expressiveness, etc...
 - Allow weaknesses and limited features for other characteristics



Towards Non-Relational Systems

- Problems with the **Impedance Mismatch**
 - Databases and Programming Languages
 - Necessity for new Data Models?
- Change in system design philosophy: **Application Database vs. Central Databases**
 - Service Oriented Architecture and Web Services

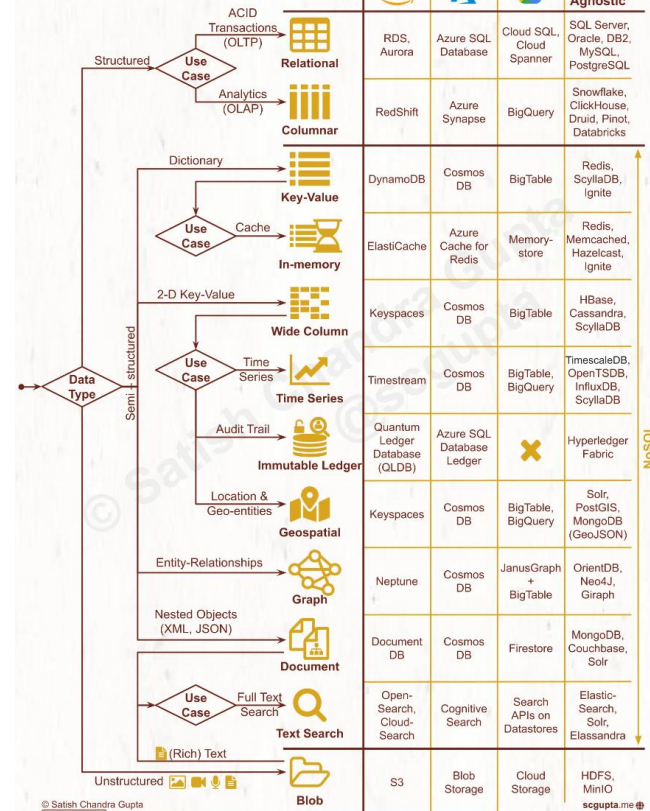


NoSQL & NewSQL

Classification	System
Key-Value Cache	Coherence, eXtreme Scale, GigaSpaces, Hazelcast, Infinispan, JBoss Cache, Memcached, Repcached, Terracotta, Velocity
Key-Value Store	Flare, Keyspace, RAMCloud, SchemaFree, Oracle NoSQL
Key-Value Store - Eventually consistent	DovetailDB, Dynamo , Dynamite, MotionDb, Voldemort, SubRecord, Riak
Key-Value Store - Ordered	Actord, Lightcloud, Luxio, MemcacheDB, NMDB, Scalaris, TokyoTyrant
Tuple Store	Apache River, Coord, GigaSpaces
Object Database	DB4O, Perst, Shoal, ZopeDB,
Document Store	Clusterpoint, CouchDB , MarkLogic, MongoDB , XML-databases
Wide Columnar Store	BigTable , Cassandra , HBase , Hypertable, KAI, KDI, OpenNeptune, Qbase
Array Databases	SciDB, PostGIS, Oracle GeoRaster, Rasdaman
Stream Databases	StreamSQL, STREAM, AURORA
Analytical Column Stores	Vertica, SybaseIQ
High Throughput OLTP	VoltDB, Hana
Graph & RDF databases	Neo4J , OpenLink Virtuoso , AllegroGraph, IBM System G, OrientDB

SQL vs. NoSQL: Cheatsheet for AWS, Azure, and Google Cloud

ml4devs.com/datastores



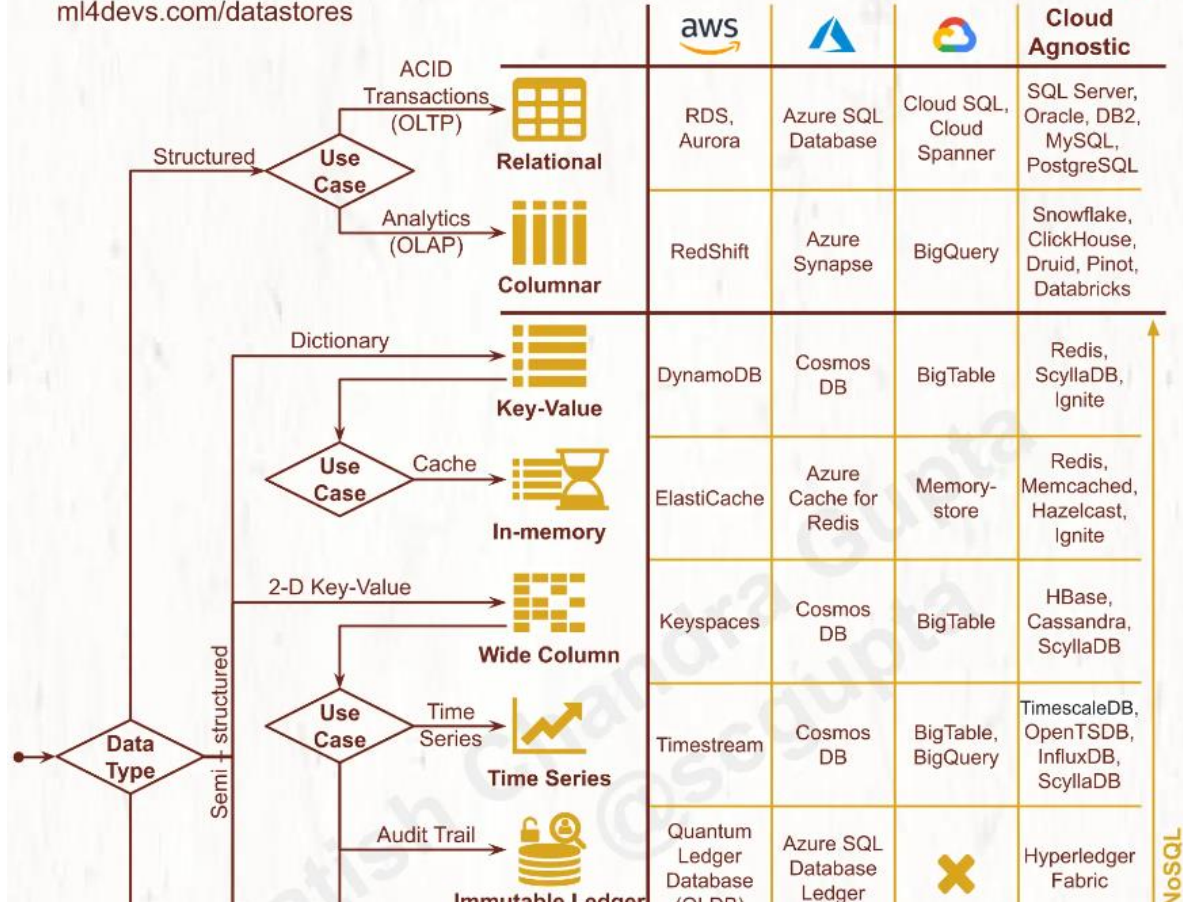
© Satish Chandra Gupta
CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/

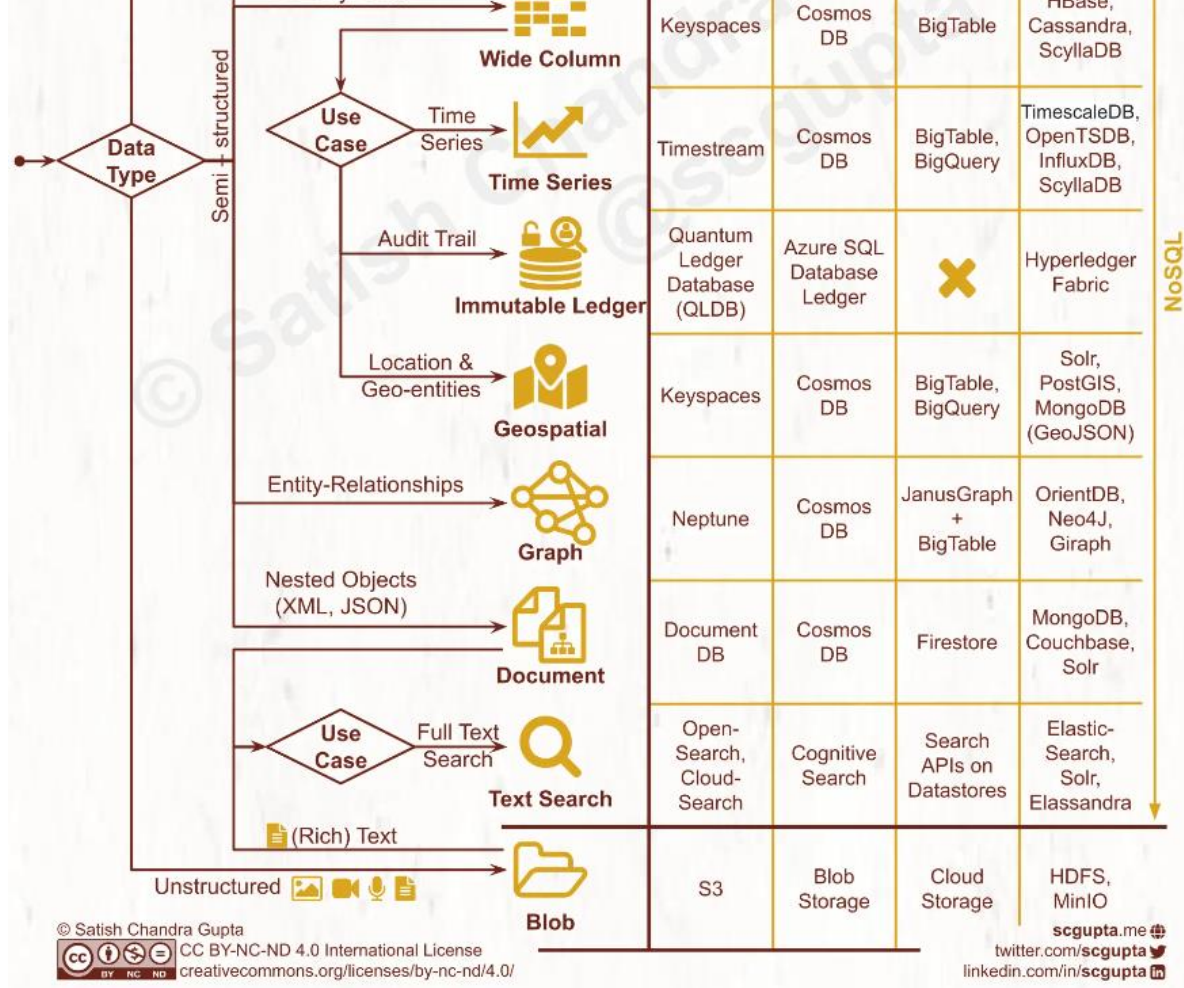
scgupta.me
twitter.com/scgupta
linkedin.com/in/scgupta

<https://www.ml4devs.com/articles/data-store-choices-sql-vs-nosql-database/>

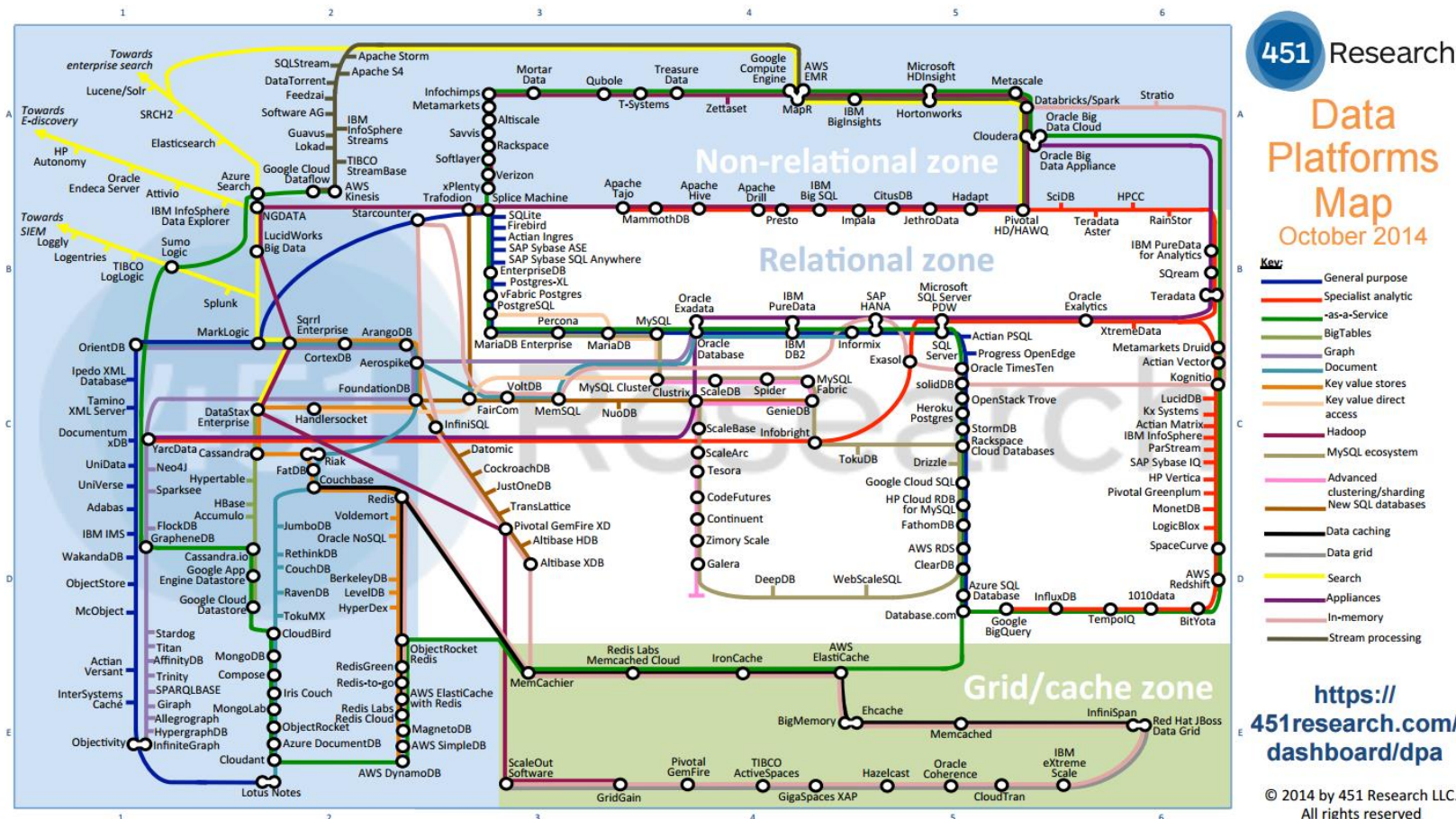
SQL vs. NoSQL: Cheatsheet for AWS, Azure, and Google Cloud

ml4devs.com/datastores





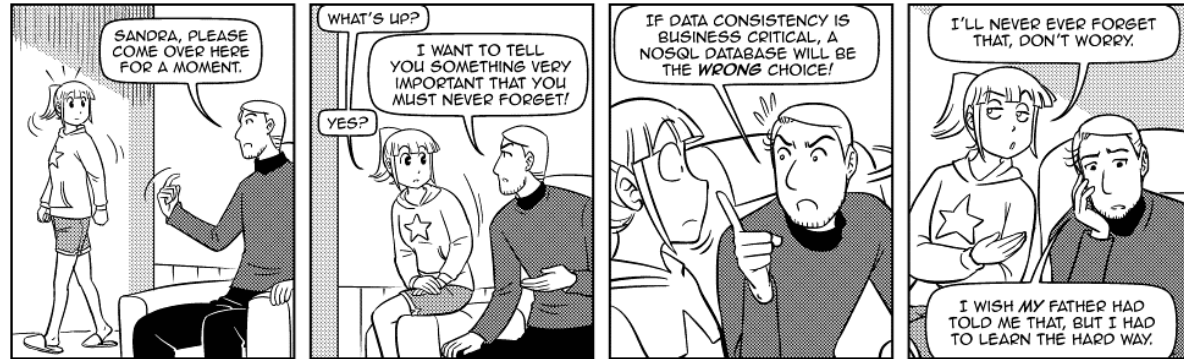
NoSQL & NewSQL



HOW TO WRITE A CV



Leverage the NoSQL boom



Sandra and Woo by Oliver Knörzer (writer) and Powree (artist) – www.sandraandwoo.com

Let's zoom in on some issues

- **Application Database vs. Central Databases**
 - Service Oriented Architecture and Web Services#
- **Impedance Mismatch**
 - Databases and Programming Languages
 - Necessity for new Data Models?
- **Distributed Storage**
 - Scaling, Clusters, and distribution (performance!)
 - ...a tiny bit today, but then also the next 3 sessions!

Application Databases vs Enterprise Databases

Application Databases

- High-scale distributed systems and databases also allow for new business models:
 - **Cloud-Computing**, computing as a utility

Three years ago CNET news:

Amazon -- that's right, Amazon -- posts big profit

The largest e-retailer usually reports slim profit as it invests in its next new thing. This time was different.

Even as operating expenses surged 25 percent during the quarter, revenue was even stronger. Sales reached \$29.1 billion, up 28 percent, while Amazon swung to a profit of \$513 million, from a loss of \$57 million the year earlier. To put that figure in perspective, Amazon posted a profit of \$596 million for all of 2015.

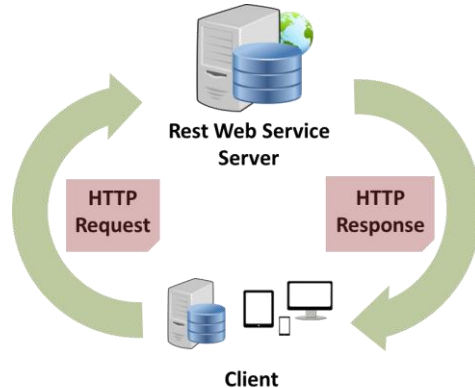
Amazon Web Services, which rents out computer server space to other companies, was a major contributor to the earnings boost. Operating income from the division reached \$604 million, up from \$195 million a year ago.

Three years ago NYtimes news:

The profits from A.W.S. represented 56 percent of Amazon's total operating income, even though the \$2.57 billion in revenue from A.W.S. — up 64 percent from a year earlier — amounted to less than 9 percent of total revenue.

Application Databases

- ...but a lot changed in 2000-2005!
 - The rise of **Service Oriented Architecture (SOA)** and **Web Services!**
 - Amazon is one of the pioneers of SOA, and self-proclaimed SOA evangelist!



The core components which make up an SOA implementation

Application Databases

- Core Idea of SOA:
 - Shared application functionality is exposed via **Web Services** to the whole company
 - Clearly defined **interfaces** and **service level agreements**
 - Complex workflows can be composed by combining multiple web services
 - Web Services usually communicate using **XML** or **JSON**
 - Usually, there is a web service bus managing communication
 - **Especially, no direct data access to other applications is necessary**
 - Just one application per database is possible!

Amazon SOA

Detour!

- **Amazon infrastructure**
 - Amazon uses a fully **service oriented architecture**
 - Each function used in any Amazon system is encapsulated in a service
 - i.e. shopping cart service, session management service, render service, catalog service, etc.
 - Each service is described by a service level agreement
 - Describes exactly what the service does
 - Describes what input is needed
 - Gives **quality guarantees**

Amazon SOA

Detour!

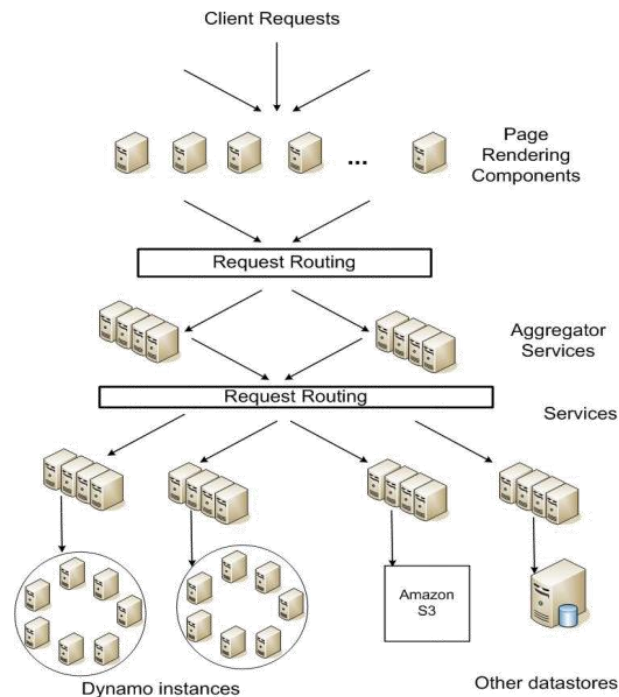
- Jeff Bezos mandate in 2002 to all Amazon Staff
 - All teams will henceforth all their data and functionality through service interfaces
 - Teams must communicate with each other through these interfaces
 - No other inter-process linking is allowed
 - No direct linking
 - No direct reads to other team's data (!!)
 - No shared memory
 - All service interfaces must be designed to be externalized
 - They should be build so that they could be shared with the world



Amazon SOA

Detour!

- Services usually use other services
 - e.g. the page render service rendering the Amazon personalized start accesses roughly 150 simpler services
 - Services may be **stateful** or **stateless**
 - **Stateless:** Transformation, Aggregation, etc.
 - **Stateful:** Shopping cart, session management, etc.
 - **Dynamo** is a data storage service which mainly drives stateful services
 - Notably: shopping cart and session management
 - There are also other storage services



Amazon SOA

Detour!

- **Service Level Agreements (SLA)** are very important for Amazon
 - Most important: **latency requirements**
 - Goal: 99.9% of all users must have an internal page render response times below 300ms
 - Not average response times, but guaranteed maximum latency for nearly all customers!
 - It should not matter what the user does, how complex his history is, what time of day it is, etc.
 - Most lower-tier services have very strict (and even tighter) SLA requirements
 - Final response is generated by aggregating all service responses
 - e.g. often, response times below 1ms for deep core services

Amazon SOA

Detour!

- **Service Level Agreements (SLA)** are very important for Amazon
 - Most important: **latency requirements**
 - Goal: 99.9% of all users must have an internal page render response times below 300ms
 - Not average response times, but guaranteed maximum latency for nearly all customers!
 - It should not matter what the user does, how complex his history is, what time of day it is, etc.
 - Most lower-tier services have very strict (and even tighter) SLA requirements
 - Final response is generated by aggregating all service responses
 - e.g. often, response times below 1ms for deep core services

Amazon SOA & The Cloud *Detour!*

- Web Services can also be exposed beyond company borders!
 - This gives rise to the cloud computing business model!



Cloud Storage

- **Cloud storage** gained momentum with to advent of **web age applications**
 - Most notable pioneers are Google, Amazon, and Yahoo
- **The main problem:**
 - “Traditional” storage solutions could not keep up with the high demand wrt. to **throughput, latency, and storage space**
 - Not enough performance & storage space
 - Too expensive
 - Too inflexible
 - Many features not necessary



History: Cloud Storage

- **Web Age applications** may grow extremely fast
 - Users, page views, and data
 - Example: **Farmville** has reached more than 150M users within 3 month
 - March 2010
 - Ratio reads : writes = 3:2
 - THIS SHOCKED PEOPLE BACK THEN!



FACEBOOK APPLICATION LEADERBOARD

Sorted by Monthly Active Users

	Name	DAU	MAU	Daily Growth
1.	FarmVille	29,716,810	81,125,786	0.34
2.	Birthday Cards	3,169,901	47,472,435	0.20
3.	Static FBML	5,878,938	32,640,373	5.91
4.	Café World	9,203,151	30,592,937	0.19
5.	Facebook for iPhone	15,437,989	28,149,229	0.19
6.	Texas HoldEm Poker	6,084,068	27,066,501	0.29
7.	Happy Aquarium (BETA)	5,028,702	26,276,826	-2.37
8.	Mafia Wars	6,770,254	25,287,875	0.12
9.	FishVille	5,855,189	25,050,350	0.29
10.	Causes	1,342,791	23,990,693	0.51
11.	Slide FunSpace	1,061,576	21,726,671	1.99

Daily active users!

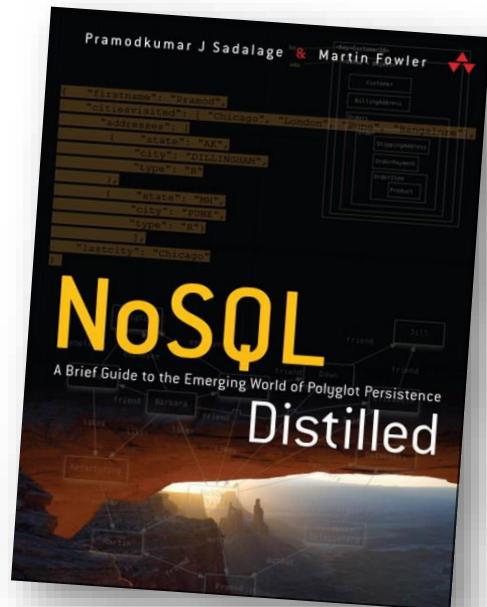
Aggregate Datastructures

Aggregate Data Structures

- As we have seen, the level of aggregation of data plays an important role in data models
 - The relational model encourages strong decomposition via normalization
 - Most NoSQL systems tend to work with **aggregates**
 - Exception: Graph databases which decompose EVERYTHING!
- Spoiler Alert:
 - We love aggregated data models in **distributed settings** as it makes querying and transaction management much easier!

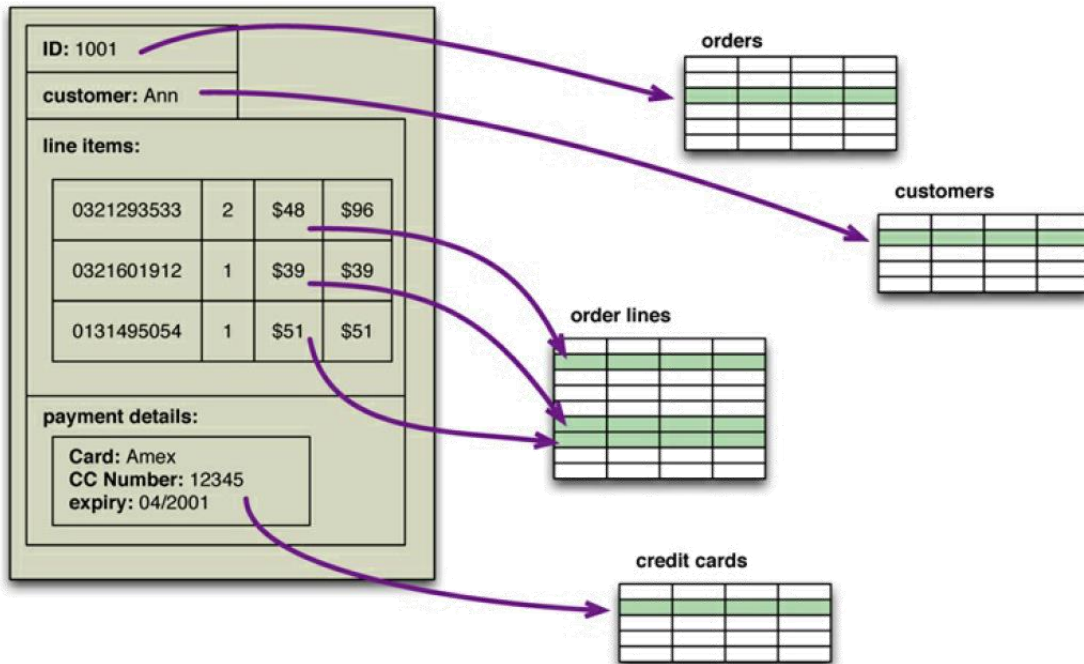
Aggregate Data Structures

- Book used for the following examples
 - NoSQL Distilled
 - Pramodkumar Sadalage & Martin Fowler



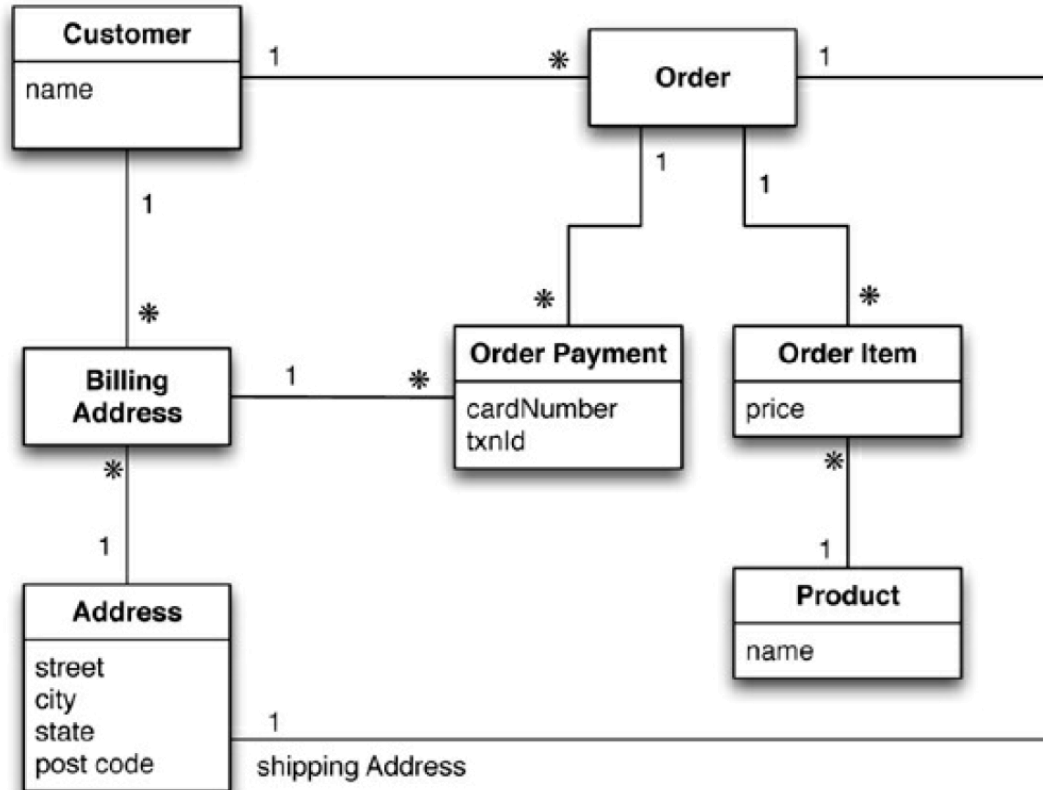
Aggregates in the Relational Model

- Aggregate Order Data decomposed to relations



Aggregates & Relational Model

UML-Style Entity
Centric Conceptual Model



Aggregates & Relational Model

Resulting Relations

Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

BillingAddress		
Id	CustomerId	AddressId
55	1	77

OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

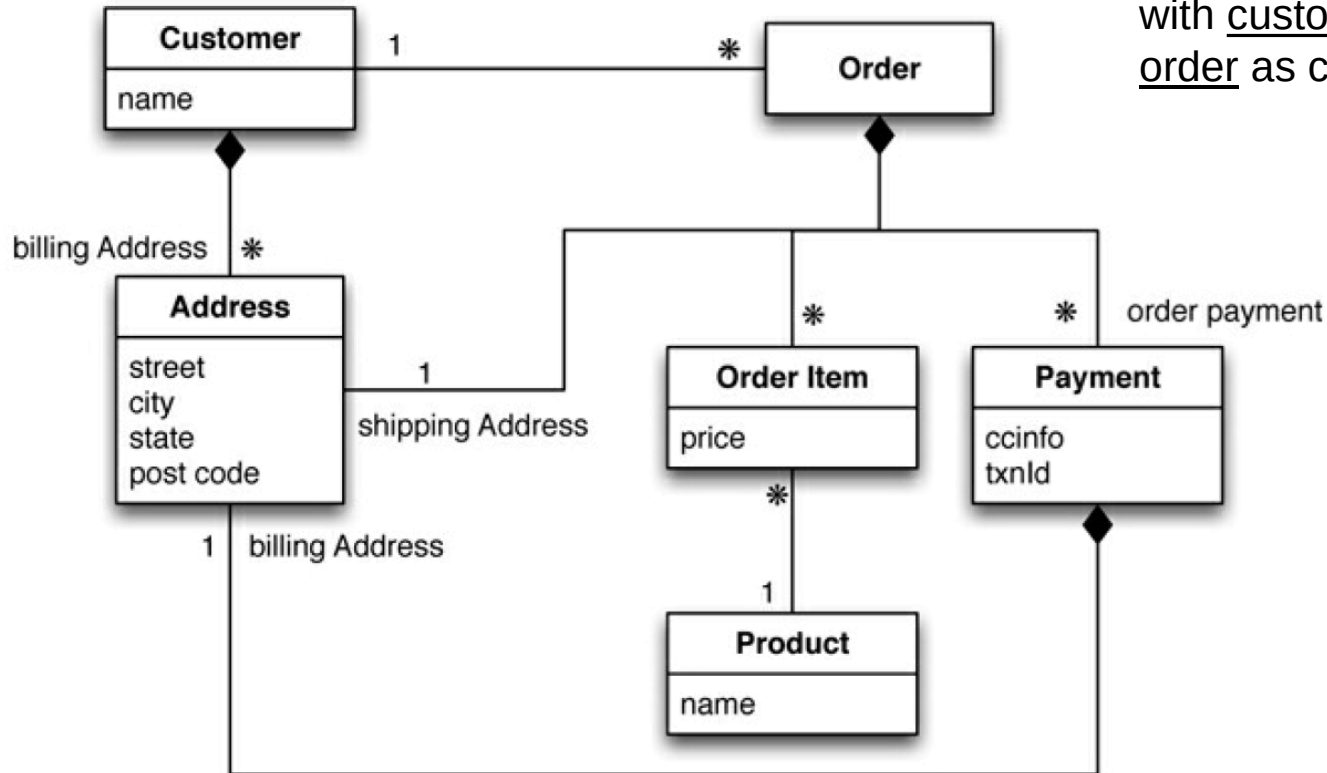
OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Aggregates & Relational Model

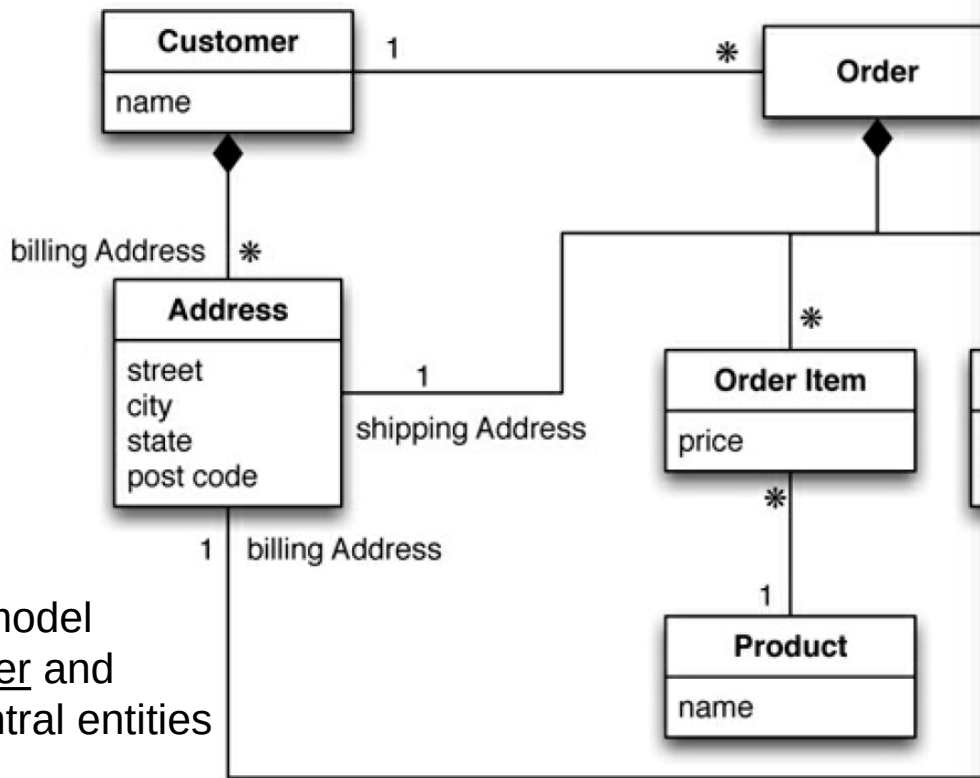
- In aggregate data models, conceptual entity types are embedded in each other
 - From one entity-relationship model, a multitude of aggregate models can be derived
 - Depends on application and point-of-view

Aggregates & Relational Model

Aggregate model with customer and order as central entities



Aggregates & Relational Model



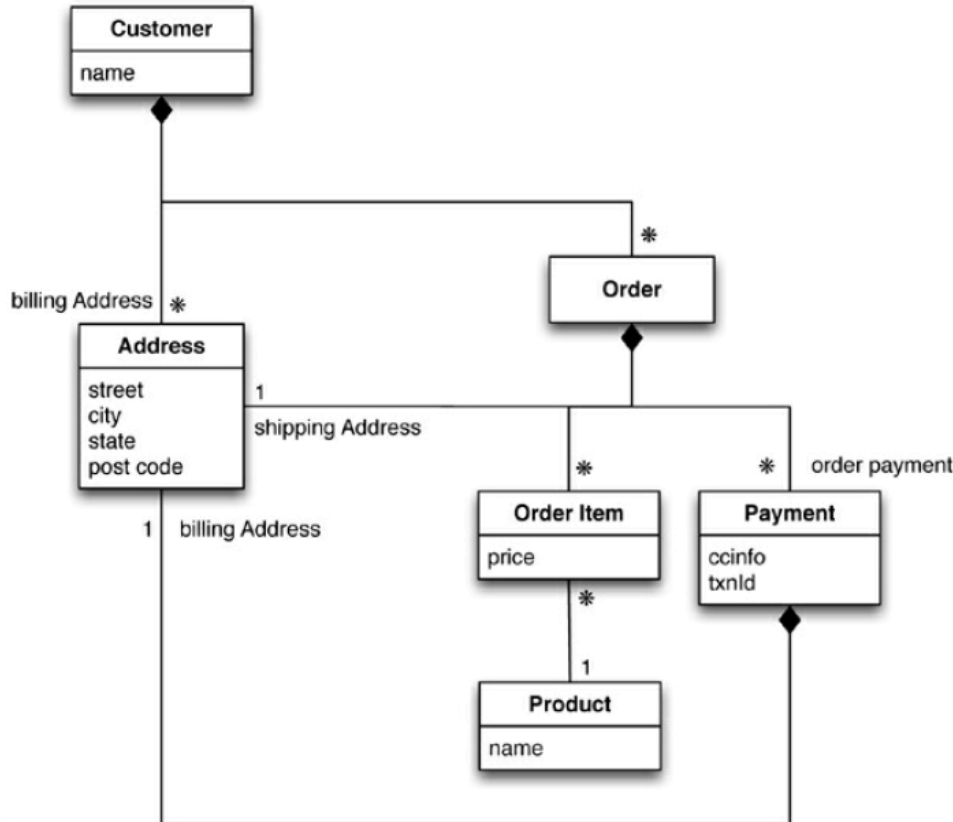
Aggregate model
with customer and
order as central entities

```
// in customers
{
  "id":1,
  "name":"Martin",
  "billingAddress":[{"city":"Chicago"}]
}

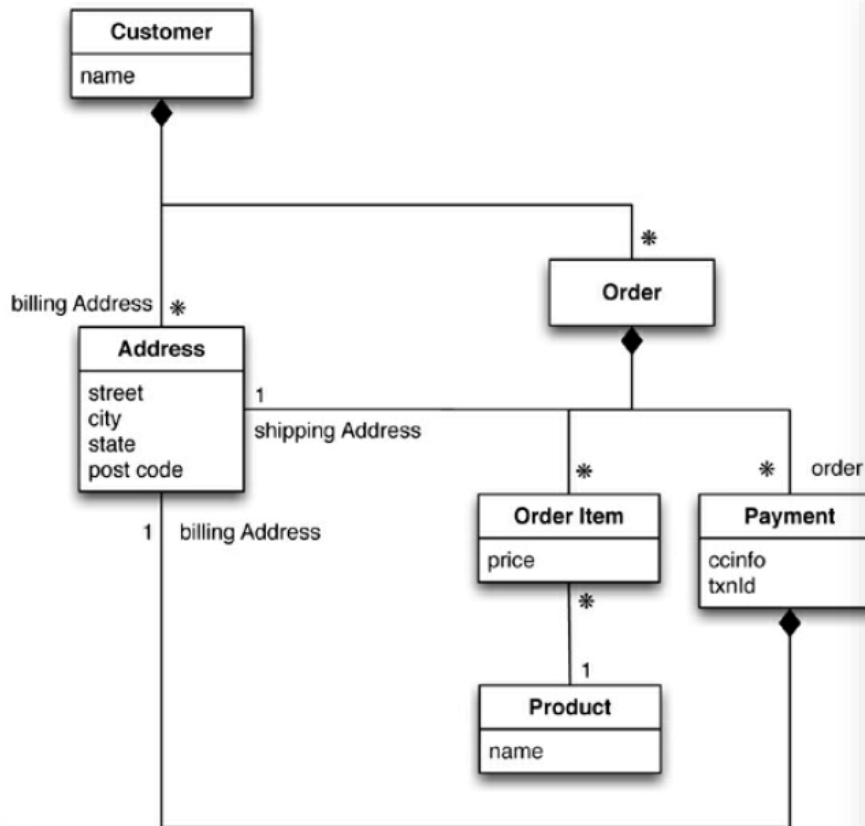
// in orders
{
  "id":99,
  "customerId":1,
  "orderItems":[
    {
      "productId":27,
      "price": 32.45,
      "productName": "NoSQL Distilled"
    }
  ],
  "shippingAddress":[{"city":"Chicago"}]
  "orderPayment":[
    {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft",
      "billingAddress": {"city": "Chicago"}
    }
  ],
}
```

Aggregates & Relational Model

Aggregate model
with customer as only
central entity



Aggregates & Relational Model



```
// in customers
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      },
      {
        "id": 100,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      }
    ]
  }
}
```

Consequences of aggregate-orientation

- No aggregates (e.g., relational model):
 - Aggregation relationships are expressed by **foreign key constraints**
 - There is no inherent difference between normal associations and aggregation when using foreign keys
 - Knowledge about aggregates remains unused
 - Using **joins**, data from different relationship types can be **flexibly combined**, creating different views dynamically
 - May cost performance
 - **Very** flexible queries possible
 - Possibility to derive any aggregate combination on the fly
 - See data warehousing
 - Relational model allows for easy implementation if transactions

Consequences of aggregate-orientation

- Using aggregates:
 - Efficient retrieval of aggregates using the aggregate identifier
 - All relevant data is part of the aggregate
 - Can be quite inflexible wrt. to queries
 - The point of view is fixed during design time; hard to represent aggregated data with a different aggregation scheme
 - Not all attributes are easily accessible in queries
 - Transaction support spanning multiple aggregates is hard to realize
 - Transactions within a single aggregate are usually fine

Aggregate Data Models

- **Key-Value models** and **document-centric models** are a direct result of aggregate data modelling
 - Both deal with aggregated data structures, usually identified by an **id**
 - Key-Value stores use opaque aggregates
 - Only the id (key) is useable in queries, the aggregate structure is unknown to the DB
 - Document-Centric stores have transparent aggregates
 - Structure of aggregates is known during query time
 - Sometimes even during design time!
 - Properties of stored on deeper aggregate levels can be used in queries

Aggregate Data Models

- Key-Value Store
 - “Retrieve object with id 1”
 - “Store all this data as object 1”
- Document Store
 - “Retrieve all customers with shipping address city in Chicago”
 - “Update price of all products in orders in customers with id 27”

```
// in customers
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "orders": [
      {
        "id": 99,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{"city": "Chicago"}]
      },
      {
        "ccinfo": "1000-1000-1000-1000",
        "txnId": "abelif879rft",
        "billingAddress": {"city": "Chicago"}
      }
    ]
  }
}
```

Key-Value

```
# Customer object
{
  "customerId": 1,
  "customer": {
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "payment": [{"type": "debit", "ccinfo": "1000-1000-1000-1000"}],
    "orders": [{"orderId": 99}]
  }
}

# Order object
{
  "customerId": 1,
  "orderId": 99,
  "order": {
    "orderDate": "Nov-20-2011",
    "orderItems": [{"productId": 27, "price": 32.45}],
    "orderPayment": [{"ccinfo": "1000-1000-1000-1000",
      "txnId": "abelif879rft"}],
    "shippingAddress": {"city": "Chicago"}
  }
}
```

key

value

<Key=CustomerID>

<Value = Object>

Customer

BillingAddress

Orders

Order

ShippingAddress

OrderPayment

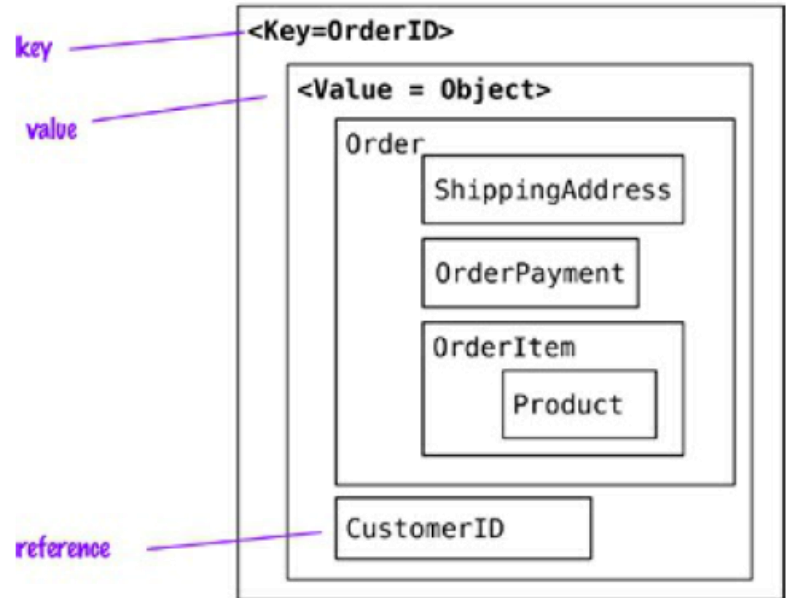
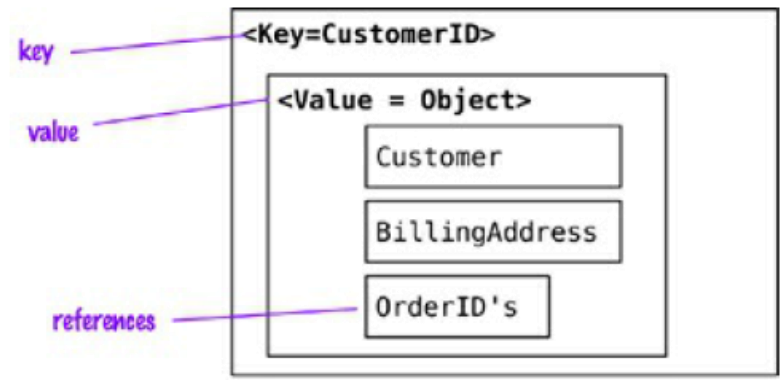
OrderItem

Product

Key-Value

```
{
  "itemid":27,
  "orders":{"99,545,897,678}
}
{
  "itemid":29,
  "orders":{"199,545,704,819}
}
```

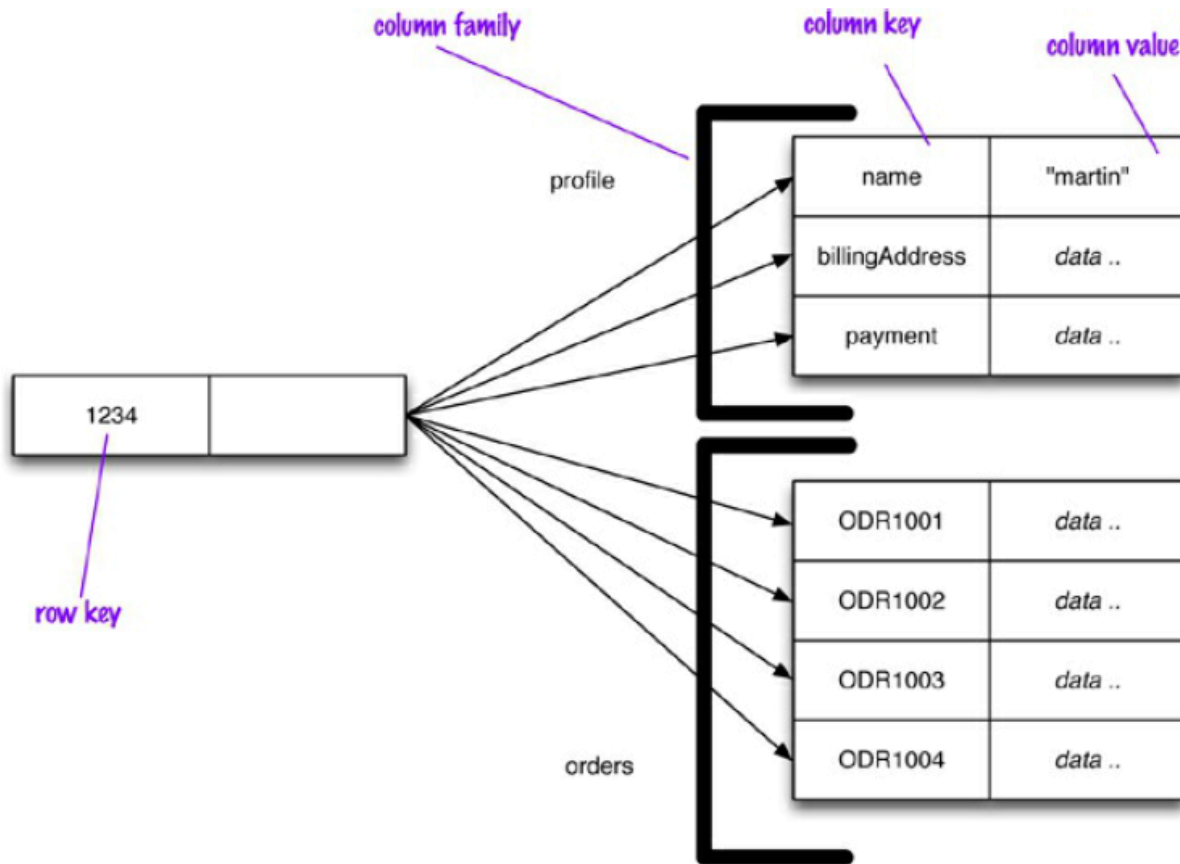
```
# Customer object
{
  "customerId": 1,
  "name": "Martin",
  "billingAddress": [{"city": "Chicago"}],
  "payment": [
    {"type": "debit",
     "ccinfo": "1000-1000-1000-1000"}
  ]
}
# Order object
{
  "orderId": 99,
  "customerId": 1,
  "orderDate": "Nov-20-2011",
  "orderItems": [{"productId":27, "price": 32.45}],
  "orderPayment": [{"ccinfo": "1000-1000-1000-1000",
    "txnId": "abelif879rft"}],
  "shippingAddress": {"city": "Chicago"}
}
```



Column-Family Models and Stores

- Sometimes also known as **wide-columnar stores** or **column stores**
 - Column stores is a bad name here, it actually means something else...
 - Basic data structure is a single **huge sparse table**
 - Each row is usually defined by a row key
 - A row contains all aggregate data relevant to the central entity it represents
 - During design time, usually only column families are defined
 - Semantically related group of columns
 - Better think of it as a sparse multi-map
 - The “rows” can grow really wide and really sparse
 - Transactions usually cover only a single row (aggregate)

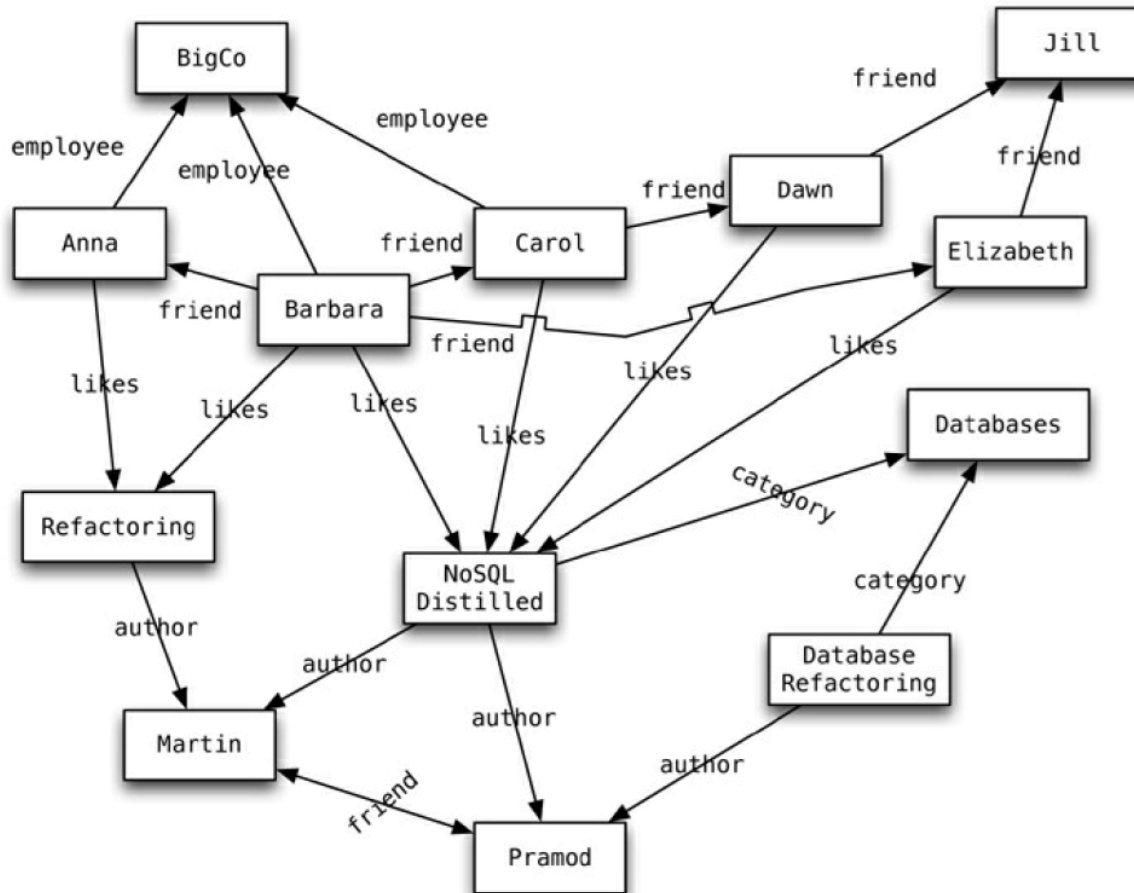
Column-Family Models



Graph Model

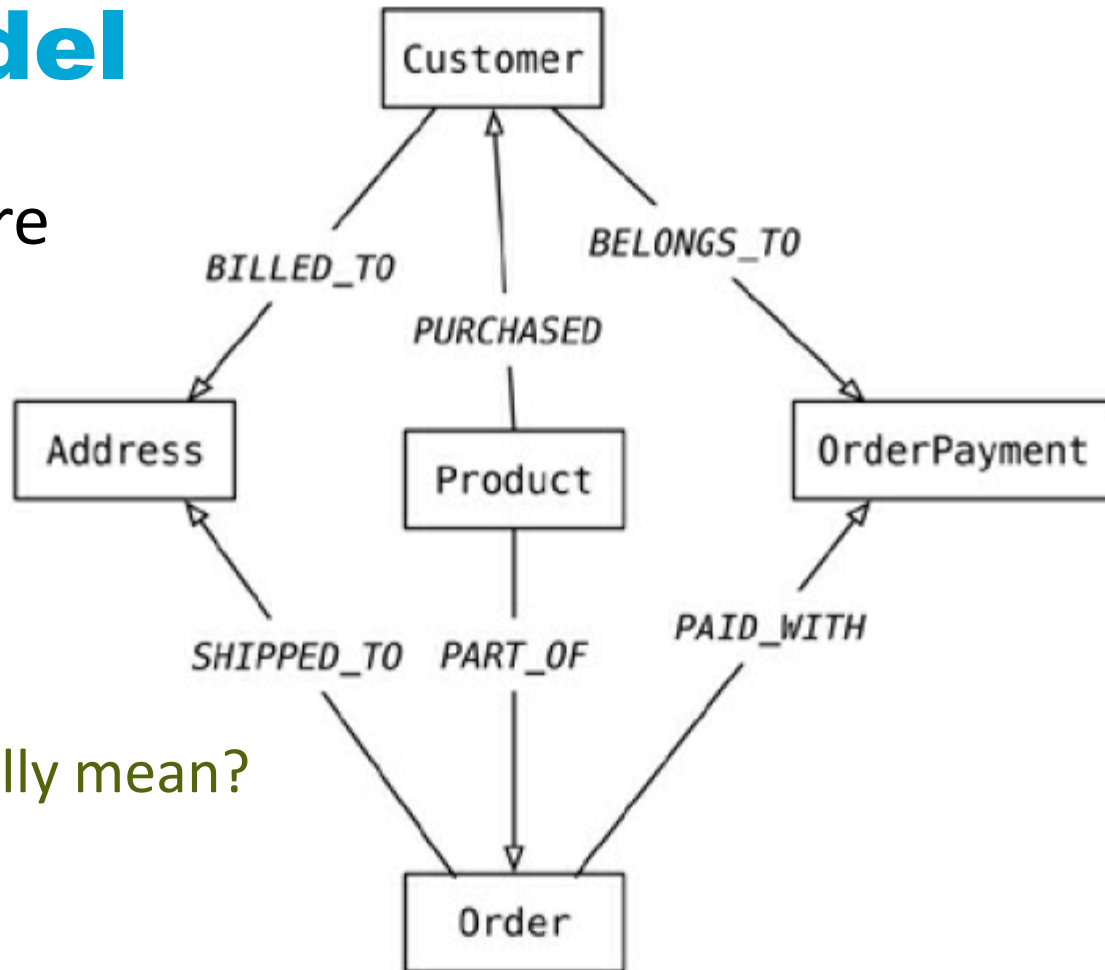
- Graph models take the opposite approach
 - **Decompose EVERYTHING!**
- Represent data as a big schemaless graph
 - Entities and properties are nodes
 - Edges represent relationships of any kind
- Extremely good for representing data that is in graph form naturally
 - Ontologies, Social Network Data, etc.

Graph Model



Graph Model

- All semantics are encoded in node and edge labels
- Big challenge:
 - What do nodes and edges actually mean?



Flexible Schemas and Schemaless DBs

- Is using schemaless databases a good idea?
 - Maybe...
 - Less worrying necessary during design time
 - Integrates well with Agile Methods like Scrum
 - Still, application will have some implicit schema!
 - User interfaces and web service interface imply a schema!
 - Dynamically building interfaces?

```
//pseudo code
foreach (Record r in records) {
    foreach (Field f in r.fields) {
        print (f.name, f.value)
    }
}
```

Build interfaces like this???

Scaling out

Distributed Databases?

- Most Relational Databases are designed to run on a single machine or a small cluster
- Challenge: **What do we do when we are faced with running a globe spanning web application?**
 - **Main topics of the upcoming weeks!**

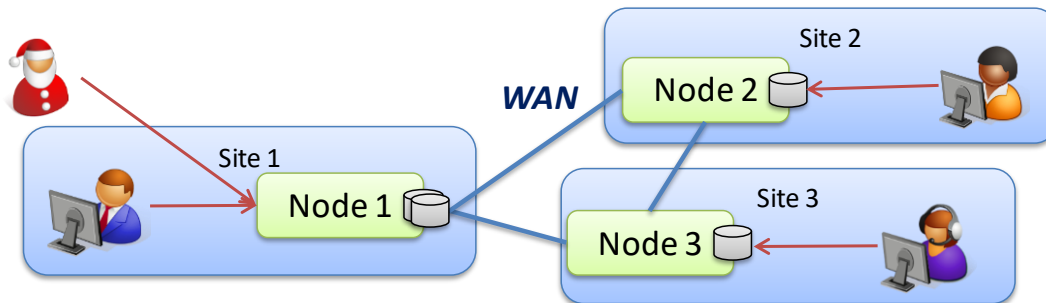


Distributed Databases?

- DRDBMSs maintain all these features in a **distributed environment**
 - Distributed **R**elational **D**atabase **M**anagement **S**ystem
 - The area of DRDBMS is thus within the area of **databases** and **computer networks**
- Potential additional **advantages** over normal RDBMS
 - Increased **robustness** vs. hardware or site failures
 - Increased **performance** due to parallelization and load balancing
 - Scalable **storage capacity**
 - Easy **access** to several shared **data sources** without complex integration

Distributed Databases?

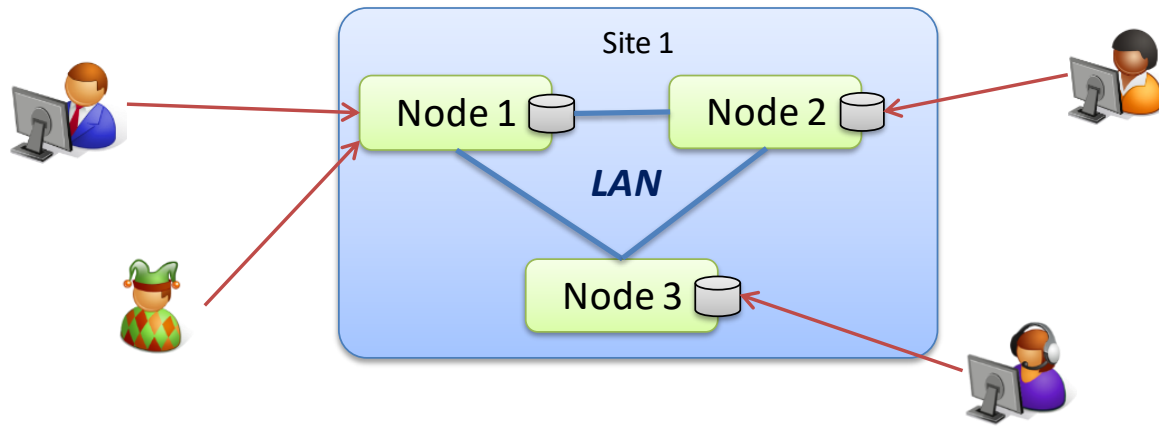
- **Homogeneous Distributed Databases**
 - **Regular Distributed Database:**
 - Each node is controlled by the same DDBMS
 - Globally shared schema
 - Nodes are linked via **slow** wide area network
 - Nodes are semi-autonomous
 - Data is usually manually distributed between the sites
 - Typical examples:
 - Databases distributed over several branches / sites (e.g. banking, insurance companies, etc.)



Distributed Databases?

– Parallel Distributed Database:

- Each node is controlled by the same DDBMS
 - Globally shared schema
- Nodes are linked via **high-speed** local area network
 - DDBMS distributes data automatically among nodes
 - Nodes show **no autonomy**
- Typical examples:
 - High performance data center

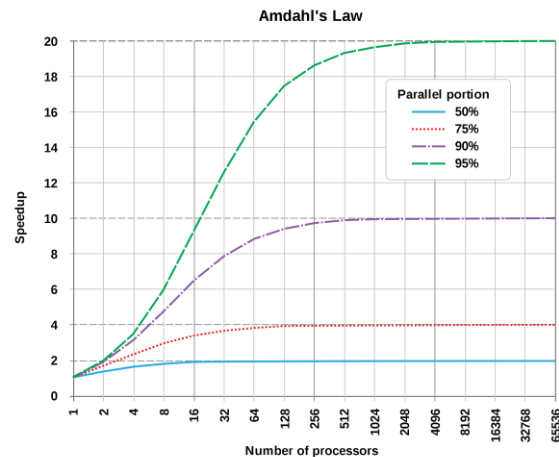


Distributed Databases?

- Distributed DBs offer solid and valuable **features**
- However, these features come at a cost
 - **Limited scaling**
 - Distributed DBs rarely scale over ~50 nodes
 - **Limited flexibility** and high **administration costs**
 - Many design decisions have are **manually** performed
 - **Adding** and **removing** nodes is tedious
 - Schemas are usually fixed or hard to change
 - **High hardware costs**
 - High-performance DDBMS require expensive **specialized** and **reliable hardware**

Distributed Databases?

- Why do we have scalability issues?
- **Amdahl's law**
 - The theoretical speedup of any distributed systems is inversely limited by the amount of code which needs to be executed in a single-threaded fashion

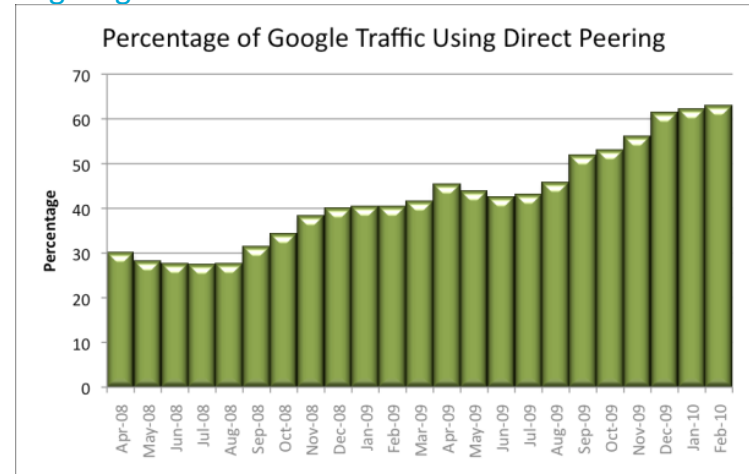
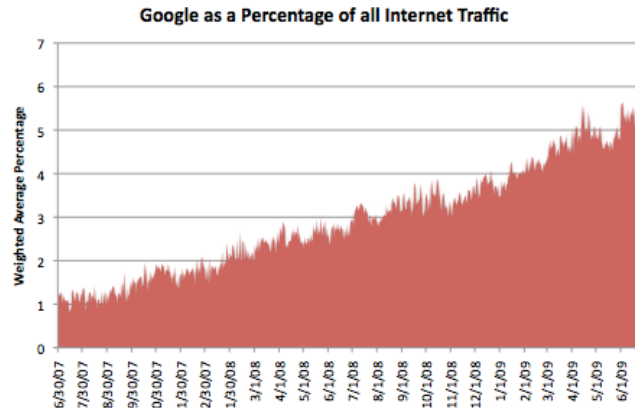


Distributed Databases?

- **Single-threaded code?!**
 - We need single-threaded code to avoid inconsistencies in data and execution
- Thus, our goal should be:
 - Parallelize as much as possible without relying on any hard semaphores or locks
 - e.g., by using strong sharding, or simply not caring about consistency...

Google Servers

- Not much is known about Google Infrastructure, but there was a paper published in 2009
- Data centers are connected to each other and major internet hubs via massive fiber lines (2010)
 - ~7% of all internet traffic is generated by Google
 - ~60% of that traffic connects directly to consumer networks without connecting to global backbone
 - If Google was an ISP, it would be the 3rd largest global carrier



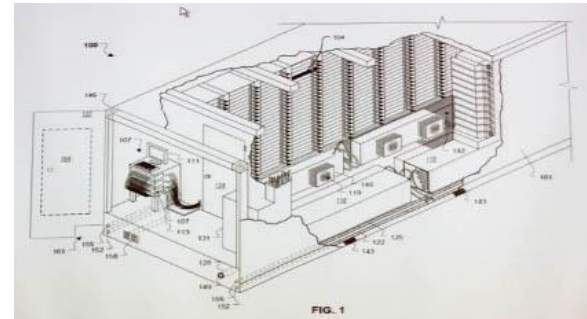
Google Servers

- Some Google Datacenter facts & rumors
 - In 2007, four new data centers were constructed for **600 million dollars**
 - Annual operation costs in 2007 are reported to be **2.4 billion dollars**
 - An average data center uses 50 megawatts of electricity
 - The largest center in Oregon has an estimated use of over 110 megawatts
 - The whole region of Den Haag is estimated to use up roughly 225 megawatts



Google Servers

- Each server rack holds 40 to 80 **commodity-class x86 PC servers** with custom Linux (2010)
 - Servers run **outdated hardware**
 - Each system has its own 12V battery to counter unstable power supplies
 - No cases used, racks are setup in standard shipping containers and are just wired together
 - More info: <http://www.youtube.com/watch?v=Ho1GEyftpmQ>



Google Servers

- Google servers are very **unstable**
 - ... but also very cheap
 - High “bang-for-buck” ratio
- Typical **first year for a new cluster** (several racks):
 - ~0.5 **overheating** (power down most machines in <5 mins, ~1-2 days to recover)
 - ~1 **PDU** (power distribution unit) **failure** (~500-1000 machines suddenly disappear, ~6 hours to come back)
 - ~1 **rack-move** (plenty of warning, ~500-1000 machines powered down, ~6 hours)
 - ~1 **network rewiring** (rolling ~5% of machines down over 2-day span)



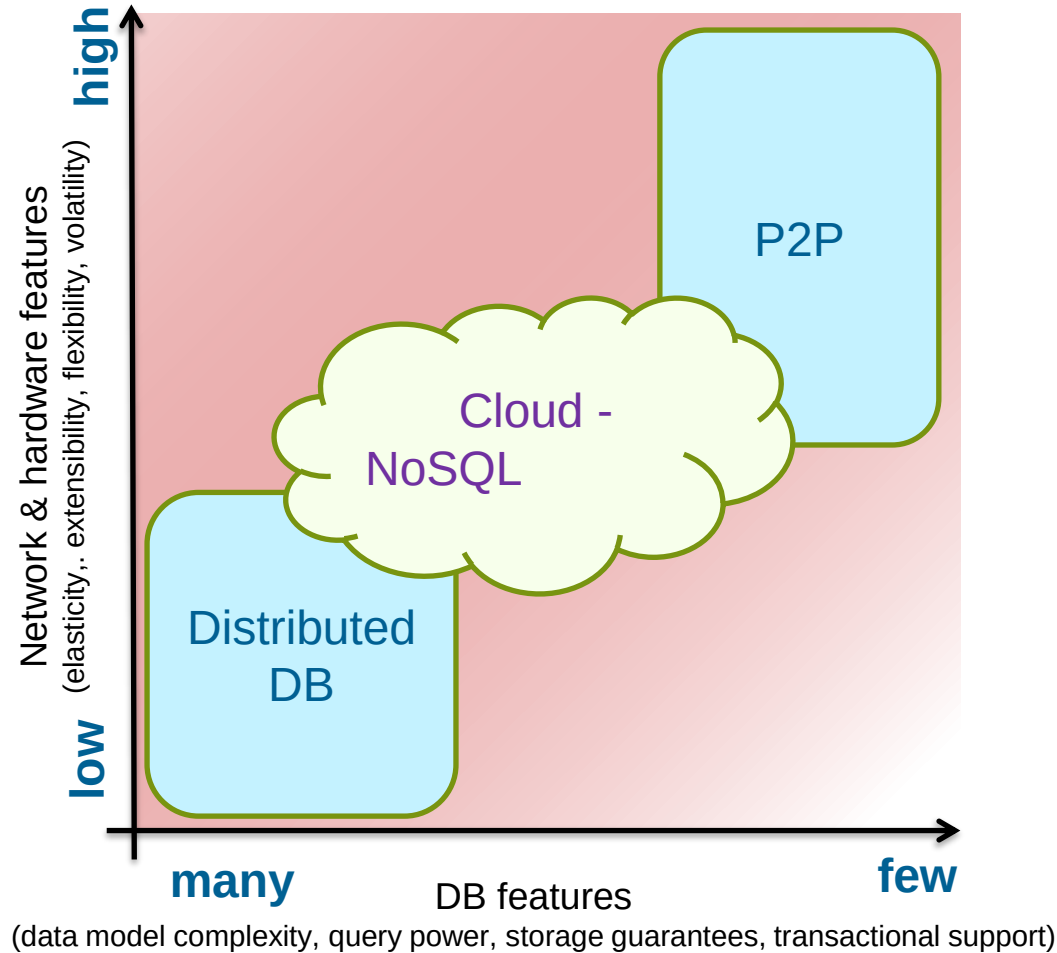
Google Servers

- ~20 **rack failures** (40-80 machines instantly disappear, 1-6 hours to get back)
- ~5 racks **go wonky** (40-80 machines see 50% packet loss)
- ~8 **network maintenances** (might cause ~30-minute random connectivity losses)
- ~12 **router reloads** (takes out DNS and external VIPs for a couple minutes)
- ~3 **router failures** (traffic immediately pulled for an hour)
- ~dozens of minor 30-second **DNS blips**
- ~1000 **individual machine failures**
- ~thousands of **hard drive failures**
- Countless **slow disks, bad memory, misconfigured machines, flaky machines**, etc.

Google Servers

- Challenges to the data center software
 - Deal with all these hardware failures while avoiding any **data loss** and **~100% global uptime**
 - Decrease **maintenance costs** to minimum
 - Allow flexible **extension** of data centers
 - Solution:
 - Use cloud technologies
 - **GFS (Google File System) and Google Big Table Data System**
 - Now, replaced by Spanner
 - To be discussed soon





Summary

- Relational Databases ARE still very good
 - ...but they are jacks-of-all-trades
 - We do have problems:
- **Impedance Mismatch**
 - Often, the relational model does not fit the data model of the application



Summary

- **Application DB vs Integration DB**
 - One of the strong points of relational DBs is that they can be used to integrate multiple applications
 - Clearly defined schemas
 - Universally supported interfaces
 - With the rise of modern SOA architecture, this feature is not that important anymore!
 - Application DBs can be lighter, more flexible, and more performant
- **The problem of Scale**

Summary

- Towards **NoSQL**
 - Name is an accidental neologism
 - Usually, it means a DB that is
 - Not using the relational model
 - Usually, aggregate models are used (often in a very simplified way)
 - Some, much more complex data models are used (XML, Logic-based, objects, etc.)
 - » Either complex aggregate models or completely normalized models like graphs
 - That runs well on clusters (but sometimes not)
 - That is open-source (or not...)
 - That is schemaless or has very flexible schemas
 - Is newly designed with modern Web standards in mind



Summary

- Not using strict **ACID** transactions
 - No transactions at all or weaker transaction models
 - See discussion about CAP Theorem and BASE transactions
- Not using **SQL**
 - Usually using simpler query paradigms
 - ...or much more complicated ones for graph / triple stores
- Especially, not supporting “typical” **query** interfaces
 - i.e. **JDBC**
 - Offering direct access from application to storage system
- Often, system can be **cloud-based**, i.e. not installed on a local server
 - System can be managed by a 3rd party

Outlook

- Next sessions: **The Problem of Scale!**
 - Why do relational DBs not scale well?
 - What can be done about that?
 - ACID transactions vs BASE transactions
 - The CAP Theorem
 - General approaches towards somewhat-ish consistent data
 - Example Systems: Amazon Dynamo, Google Bigtable / Megastore / Spanner

See you next time!



TU Delft

Delft
University of
Technology



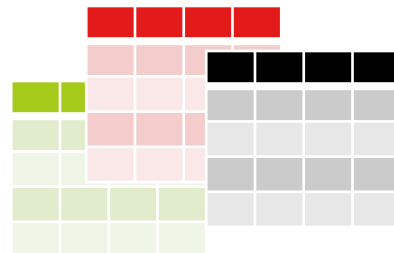
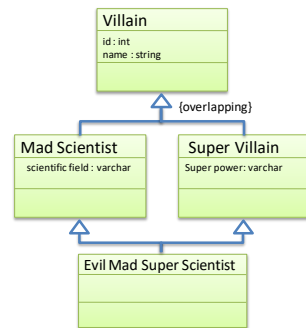
Universität
Zürich

The Impedance Mismatch

- I would have liked to talk about this topic, but there is really not enough time.
Thus, I move it herefor you to read at home.
 - This is not relevant for the exam.

The Impedance Mismatch

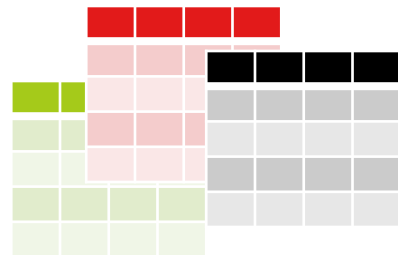
- Within typical software engineering cycles, **application data models** are developed
 - data models are used **internally** in application, following its design and programming paradigms
 - nowadays, most application data models are **object-oriented**
 - also often called **domain model**
- When developing an application using a database there always is one huge problem
 - **how do you map your domain data model to your database data model?**
 - **impedance mismatch!**



The Impedance Mismatch

- What is the **impedance mismatch**?
 - Short version: **Problems arising from programming language using a different data model than a DB**
 - **Challenge: How to map between them?**
 - **DB: Relational Model**
 - Tables with rows and columns
 - Columns have a datatype
 - There may be foreign key constraints between columns
 - All tables are “flat”
 - Simple types - no nested data structures!
 - Remember: 1st Normal Form

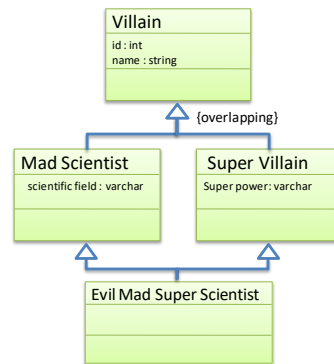
JAVA != SQL



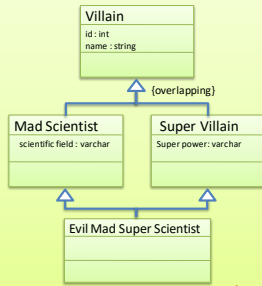
The Impedance Mismatch

– Host Language

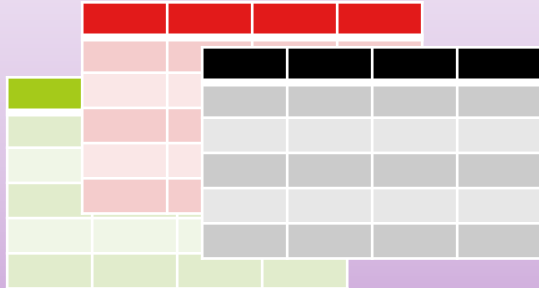
- Different data types, often no explicit NULL values
- Usually no native support for table structures compatible with DBs
- Different data models
 - **object-oriented data models**
 - record-oriented data models
- Usually, complex nested objects/records are possible!



Application Data Model



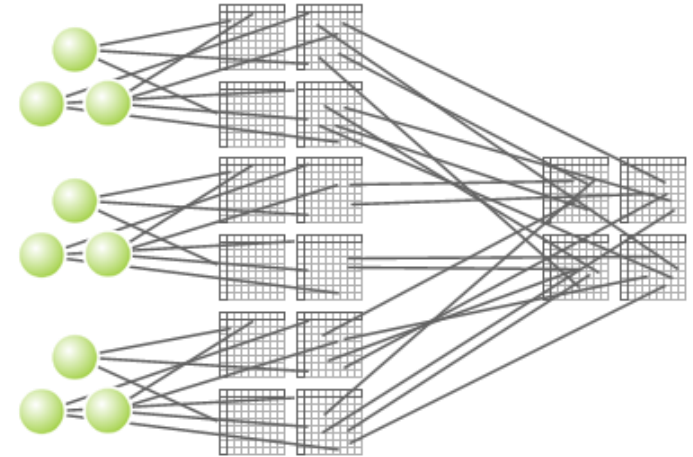
Database Data Model



Application

RDBMS

RDBMS



Object-Oriented Model

Detour!

- The object-oriented model
 - objects **represent objects in the real world**
 - thus, objects have a **state** and a **behavior**
 - e.g. *This car is **blue** and has **33 PS**, it can **drive** and **honk the horn**.*
- Besides state and behavior, objects may have **complex relationships** to other objects
 - Usually, all relationships we already know from UML are possible
 - **generalization, specialization, aggregation, composition, association, etc.**
 - **Example**
 - *A car is built of have 4 wheels and 2 doors. It can have 1 owner. A car is a subtype of vehicle. A SUV is a specialized car.*
 - *When the car drives, also the wheels are moving and rotating.*



Object-Oriented Model

Detour!

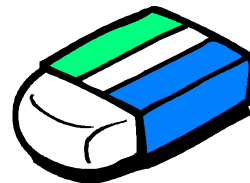
- How to store complex relationships (**object structure**) in **relational DBs**?
 - As a whole, this structure is **hard to capture** in a **RDBMS**
 - Remember: you only have tables, columns and rows
 - objects needs to be **disassembled** and stored in **multiple tables** linked by foreign key relationships
 - Think of this (Garage Metaphor):
Every time you want to park your car in the garage, you have to disassemble it and assemble it when you want to drive to work...



Example: Manual Persistence

Detour!

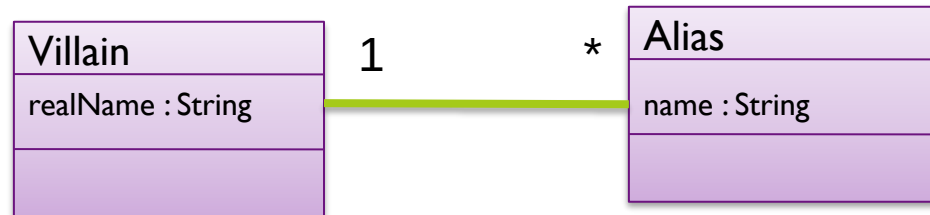
- Providing object persistence is a complex task
 - provide means to **create, read, update, and delete** persistent objects
 - called the **CRUD** operations
 - for each of these operations, respect the object's **identity** and **structure**
 - create explicit identities if necessary
 - break object structure into relationships among entities



Example: Manual Persistence

Detour!

- Of course, persistence can be achieved **manually** by using tools we already know
 - e.g. **SQL** and **Java/JDBC**
 - however, this approach often is cumbersome and takes a lot of effort
- **Example:** let's make a simple JavaBean persistent
 - Let's build a super hero/villain DB....



Example: Manual Persistence

Detour!

This is all the application actually needs!

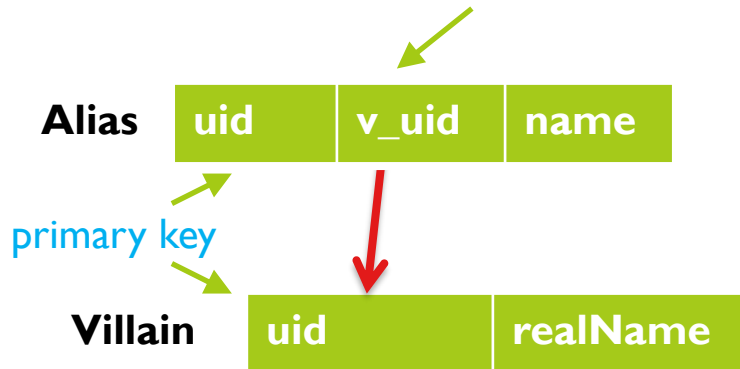
Villain.java

```
public class Villain {  
    String realName;  
    Set<Alias> aliases;  
    // some getter/setter methods  
}
```

Alias.java

```
public class Alias {  
    String name;  
    // some getter/setter  
}
```

- **Relational tables**
 - beans have **no primary keys** (names are not unique)
 - generate a key somehow...



Detour!

Villain.java

Take care of identity

```
int uid;
public void createPersistent(Connection conn) throws SQLException {
    uid = magicallyCreateAnUID(this);
    Statement stmt = conn.createStatement();
    stmt.executeUpdate(
        "INSERT INTO villains (" + uid + ", " + realName + ")"
    );
    for (Alias alias : aliases) {
        alias.createPersistent(this, conn);
    }
}
```

Take care of structure

Alias.java

```
int uid;
public void createPersistent(Villain villain, Connection conn)
    throws SQLException {
    uid = magicallyCreateAnUID(this);
    Statement stmt = conn.createStatement();
    stmt.executeUpdate(
        "INSERT INTO alias (" + uid + ", " +
        villain.uid + ", " + name + ")"
    );
}
```

Example: Manual Persistence

Detour!

- **Note:** The generated **UID** is not related to the *real* object identity in Java (i.e. the **reference ID**)
 - the **reference ID** is important e.g. to build data structures at runtime but has no semantical meaning otherwise
 - the **generated UID** is important to identify and connect rows in the DBMS and also has no semantical meaning
 - although the **UIDs** does not contain relevant information, the application **has to maintain** it



Example: Manual Persistence

Detour!

- **CRUD read operation**
 - for reading an object, you need to **know its UID**
 - for accessing the full **object structure**, you have two options
 - **eager loading:**
loads the whole object structure as soon as the base object is requested
 - good performance if you usually need the whole structure
 - **lazy loading:**
loads only the base object when requested,
loads remaining parts of the object's structure when they are needed
 - good performance if you usually only need parts of the structure

Example: Manual Persistence

Detour!

CRUD read operation (lazy)

You need the UID, somehow...

Villain.java

```
public static Villain readPersistent(int uid, Connection conn)
    throws SQLException {
    Statement stmt = conn.createStatement();
    Villain villain = new Villain();
    ResultSet rs = stmt.executeQuery(
        "SELECT * FROM villains WHERE uid=" + uid
    );
    if(rs.next()) {
        villain.uid = uid;
        villain.realName = rs.getString("realName");
        villain.aliases = null;
    } else {
        // handle case when uid does not exist
    }
    return villain;
}
```

Example: Manual Persistence

Detour!

CRUD read operation (lazy)

Villain.java

```
public Set<Alias> getAliases() throws SQLException {
    if (aliases == null) {
        aliases = new HashSet<Alias>();
        Statement stmt = ... // create statement somehow
        ResultSet rs = stmt.executeQuery(
            "SELECT * FROM alias WHERE v_uid= " + uid
        );

        while (rs.next()) {
            Alias alias = new Alias();
            alias.uid = rs.getInt("uid");
            alias.name = rs.getString("name");
            aliases.add(alias);
        }
    }
    return aliases;
}
```

Example: Manual Persistence

Detour!

- **CRUD update**
 - for updates, there are also various implementation approaches
 - **immediate updates**
 - **directly persist updates after they occur**
 - embed JDBC calls directly in each beans set methods
 - performance might be **very low**
 - only few problems with transactional consistency



Example: Manual Persistence

Detour!

- **explicit deferred updates**
 - collect updates and apply them in batches
 - every x seconds, after n updates or at the end of a bigger procedure
 - JDBC provides dedicated batch update/insert methods
- **implicit deferred updates**
 - mark object as *dirty*, when its content has been changed
 - provide an **additional** method for updating the Beans content in the DB if it is dirty
 - identify and update **dirty objects** periodically
 - **traverse** the whole **object structure recursively** and also write all dirty, related objects
- deferred updates perform better, but they may cause **problems with transactional consistency**

Example: Manual Persistence

Detour!

- Storing complex objects requires a lot of **boilerplate code**
 - The code for the data model is littered with code dealing with persistence
 - most of it is trivial and could be generated
 - makes the data model **dirty** and **inflexible**
 - In this example, the initial Villain.java model grew from **26 lines** of code to over **180 lines** of code
 - even though the implementation is **very simple**

Example: Manual Persistence

Detour!

- **Summary**
 - a considerable amount of **work is necessary**
 - there are **many fetching and update schemes** with different properties and performance impacts
 - **lazy load vs. eager load**
 - **immediate update vs. explicit update**
 - ...
 - there are several problems involved
 - **generating IDs** and propagating them
 - dealing with the **object structure**
 - keeping **transactional consistency** in multi-user scenarios
 - providing **sufficient performance**
 - how to perform **complex querying?**

Impedance Mismatch

- More general discussion:
What are naturally occurring native datatypes?